



SAKINAH

Backend Schema and API Document

AS BUILT

Version 1.0 | 07 September 2026

Verified from the secured repository. Secret values excluded.

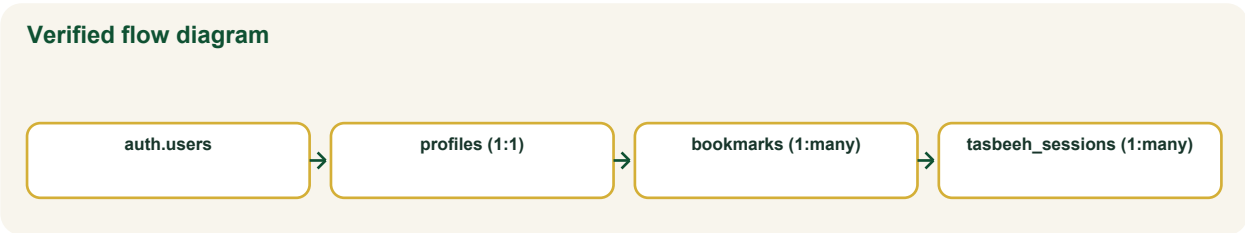
Contents

1. Backend architecture
2. Database inventory
 - `public.profiles`
 - `public.bookmarks`
 - `public.tasbeeh_sessions`
3. Relationships, ownership, lifecycle
4. Enums, indexes, seed/migrations
5. Supabase operation inventory
6. External read API inventory
7. Validation and security controls
8. Search/pagination/sort conventions
9. Error behavior
10. Upload/payment/rate limits
11. Transactions and concurrency
12. Performance and data risks
13. Sensitive fields never to return/log
14. Known gaps

Product: Sakinah | Status: As Built | Version: 1.0 | Date: 2026-09-07

1. Backend architecture

There is no repository-owned HTTP backend, controller, server runtime, ORM, Prisma schema, or server route. The browser uses Supabase Auth/PostgREST directly. Database DDL is in `supabase/schema.sql`; external Islamic content uses browser `fetch` adapters in `src/lib/islamicApi.js`.



2. Database inventory

public.profiles

Field	Type	Required	Default	Constraints
id	uuid	Yes	none	PK; FK → auth.users(id); cascade delete
name	text	Yes	empty string	none beyond not-null
email	text	No	null	none
city	text	No	null	none
preferences	jsonb	Yes	{}	not-null
created_at	timestamptz	Yes	now()	not-null
updated_at	timestamptz	Yes	now()	not-null; no auto-update trigger

public.bookmarks

Field	Type	Required	Default	Constraints
id	bigint identity	Yes	generated	PK
user_id	uuid	Yes	none	FK → auth.users(id); cascade delete
kind	text	Yes	none	one of ayah, hadith, book
reference	text	Yes	none	unique with user/kind
payload	jsonb	Yes	{}	not-null; shape/size not constrained
created_at	timestamptz	Yes	now()	not-null

Unique index/constraint: (user_id, kind, reference).

public.tasbeeh_sessions

Field	Type	Required	Default	Constraints
id	bigint identity	Yes	generated	PK
user_id	uuid	Yes	none	FK → auth.users(id); cascade delete
dhikr	text	Yes	none	not-null
count	integer	Yes	none	count > 0
created_at	timestampz	Yes	now()	not-null

3. Relationships, ownership, lifecycle

Every row belongs to one Supabase Auth user. Profile is one-to-one; bookmarks and sessions are one-to-many. Deleting an Auth user cascades to all three tables. There is no soft delete, audit table, record version, or application-defined retention schedule.

RLS is enabled for each table. FOR ALL policies require `auth.uid()` to equal `id` or `user_id` in both USING and WITH CHECK. This governs select, insert, update, and delete. No role enum or admin bypass is defined in repository SQL.

A security-definer trigger `handle_new_user()` inserts profile ID/name/email after an Auth user is created and fixes `search_path=public`. Existing profile conflicts are ignored.

4. Enums, indexes, seed/migrations

- No PostgreSQL enum type exists; `bookmarks.kind` uses a check constraint.
- Explicit index beyond primary/unique/FK backing behavior: none declared.
- There is one idempotent-oriented SQL setup file, not an ordered migration system.
- No seed script exists.
- Re-running the file may fail at `create policy` because policies are not conditionally dropped/created.

5. Supabase operation inventory

These are SDK operations, not custom HTTP endpoints.

Operation	Method/path concept	Auth	Ownership	Request/response
Get session	Supabase Auth <code>getSession</code>	Optional	Current browser session	Session/user or null
Auth-state listener	Supabase Auth channel	Optional	Current session	Auth event/session
Sign up	Auth <code>signUp</code>	Public/rate-limited externally	New account	Email/password/name → user/session/error
Sign in	Auth <code>signInWithPassword</code>	Public	Account credentials	session/user/error
Google OAuth	Auth <code>signInWithOAuth</code>	Public	Provider identity	HTTPS authorize URL
Password reset	Auth <code>resetPasswordForEmail</code>	Public	Email purpose	generic client message
Sign out	Auth <code>signOut</code>	Authenticated	Current session	completion/error
Load Ayah saves	<code>bookmarks.select</code> filtered <code>kind=ayah</code>	Authenticated	RLS	<code>reference,payload[]</code>
Upsert/delete Ayah	<code>bookmarks.upsert/delete</code>	Authenticated	RLS + supplied <code>user_id</code> on insert	row/error

Operation	Method/path concept	Auth	Ownership	Request/response
Load custom Tasbeeh	<code>bookmarks.select</code> , kind book, reference like <code>tasbeeh:%</code>	Authenticated	RLS	payload list
Save custom Tasbeeh	<code>bookmarks.upsert</code>	Authenticated	RLS	bounded JSON payload
Save Tasbeeh session	<code>tasbeeh_sessions.insert</code>	Authenticated	RLS	positive count row/error
Save/delete Hadith	<code>bookmarks.upsert/delete</code> , kind hadith	Authenticated	RLS	narration payload/error

No REST status/error wrapper is controlled by this repository; Supabase SDK/PostgREST semantics apply.

6. External read API inventory

Method	Endpoint pattern	Purpose	Auth	Validated inputs	Main failure
GET	<code>api.alquran.cloud/v1/surah</code>	Surah metadata	None	none	generic unavailable error
GET	<code>/surah/{1..114}/editions/...</code>	Text/translations/audio	None	bounded Surah	load error
GET	<code>/edition</code>	Edition list	None	response filtered	unavailable
GET	<code>/juz/{1..30}/{edition}</code>	Juz content	None	number + edition syntax	load error
GET	<code>api.aladhan.com/v1/timings/{date}</code>	Prayer times	None	lat/lon/date bounds; fixed method/school	unavailable
GET	<code>api.aladhan.com/v1/asmaAlHusna</code>	99 Names	None	none	inline reconnect error
GET	<code>cdn.jsdelivr.net/.../{path}.json</code>	Hadith collection/section/item	None	lower-case safe path syntax	unavailable
GET/embed	<code>youtube-nocookie.com/embed/video series</code>	Seerah video	None	repository-fixed playlist/index	embed failure
GET	<code>/assets/hisnul-muslim.json</code>	Dua corpus	Same origin	none	fallback list

There are no upload, webhook, payment, admin, export, or server-side URL-fetch endpoints.

7. Validation and security controls

- Auth email format/254 max, name 80 max, password 8–128 on signup.
- Surah 1–114, Juz 1–30, edition syntax up to 64 characters.
- Coordinates latitude –90..90 and longitude –180..180.
- Hadith path syntax prevents traversal/operator injection.
- Custom Tasbeeh title 80, meaning 240, target 1–100,000.
- Image JPEG/PNG/WebP, ≤1.5 MB, verified data-URL prefix.
- React escapes rendered data; no raw HTML sink found.
- RLS prevents IDOR/BOLA across declared tables when deployed as written.
- OAuth provider navigation is exact-origin/HTTPS constrained.

8. Search/pagination/sort conventions

- Quran/subject/book/Dua search is client-side after metadata/content load.
- Hadith provider sections drive topic navigation.
- Some lists expose local “load more”/pagination controls; no universal server pagination contract exists.
- No arbitrary sort-field input reaches SQL.

9. Error behavior

Provider adapters throw generic Error messages. Auth UI maps raw provider errors to generic public messages and logs redacted codes. Supabase data-operation error handling is inconsistent: Quran loading logs an error while several writes do not display a failure. Correlation/request IDs are not implemented because there is no application server.

10. Upload/payment/rate limits

Upload is limited to a custom image encoded in JSON, not a dedicated file endpoint. Server-side magic-byte checking, malware scan, and quota are absent. Payments/webhooks are not applicable. Auth/API rate limits are provider-managed and not verified in repository configuration.

11. Transactions and concurrency

No multi-statement application transaction is defined. Bookmark uniqueness plus upsert prevents duplicates for (user, kind, reference). UI read-modify-write operations can race across devices; last accepted write wins. Tasbeeh session inserts are append-only. No idempotency key is used.

12. Performance and data risks

- Primary and bookmark unique indexes support principal lookup patterns.
- Consider an explicit index on tasbeeh_sessions(user_id, created_at desc) before large growth.
- Consider bookmarks(user_id, kind, created_at desc) if queries expand beyond the unique index order.
- JSON payload/image size is the main storage-exhaustion risk.
- No N+1 server query exists; multiple direct provider calls can still increase client latency.
- Full Hadith collections can be large; provider pagination/virtualized rendering should be assessed.

13. Sensitive fields never to return/log

Passwords, password hashes, refresh/access tokens, service-role keys, OAuth/SMTP secrets, raw database URLs, and unrelated Auth metadata. Client views should use only user ID/name/email needed for profile/account behavior. Current application does not receive password hashes.

14. Known gaps

Remote RLS state, Auth rate limits, CAPTCHA, token lifetime, refresh reuse protection, SMTP, Google settings, TLS database policy, backup/retention, and production indexes are Not verified from repository. Owner verification is required; see the security audit.